

AI-Based Smart Interview System for Automated Candidate Screening using NLP, Computer Vision and PostgreSQL

Naresh Kumar B

Department of Computer Science and Engineering

Artificial Intelligence and Data Analytics

Sri Ramachandra Faculty of Engineering and Technology

Sri Ramachandra Institute of Higher Education and Research, Chennai, India

Chennai, Tamil Nadu, India

Abstract — Recruitment teams and academic placement cells frequently conduct first-level interviews for a large number of candidates. Manual screening is time-consuming and can produce inconsistent feedback when evaluators assess similar profiles repeatedly. This paper presents an AI-Based Smart Interview System that automates the initial interview assessment workflow using a Flask web application, Natural Language Processing based answer scoring, OpenCV based camera monitoring, PostgreSQL backed question management and automated report generation. The system verifies candidates using OTP, assigns role-based aptitude and programming questions, evaluates submitted answers through exact answer matching, keyword coverage, TF-IDF cosine similarity and communication scoring, and generates a structured PDF report for administrator review. The proposed system provides a low-cost and locally deployable approach for consistent candidate screening without depending on paid external AI services.

Keywords — Smart interview system; candidate screening; natural language processing; TF-IDF; OpenCV; Flask; PostgreSQL; automated assessment.

I. INTRODUCTION

Interview screening is an important stage in recruitment because it helps an organization identify candidates with suitable technical knowledge, reasoning ability, communication skill and professional behaviour. In a conventional process, evaluators ask questions, listen to responses, manually compare answers and prepare feedback. This process is useful but becomes difficult when a large number of candidates have to be screened for similar entry-level roles.

Online assessments reduce some manual effort, but many systems only support objective question scoring. They do not evaluate descriptive answers, do not provide structured feedback and do not store complete interview evidence for later review. Similarly, video interview platforms can record candidate responses, but they often require manual review or paid cloud-based analytics. Hence, there is a need for a simple, transparent and locally deployable smart interview system that combines web-based interview flow, answer evaluation, basic camera monitoring, database storage and report generation.

The proposed AI-Based Smart Interview System addresses this requirement. It supports candidate registration, OTP verification, role selection, role-wise question assignment, NLP based scoring, OpenCV based face presence monitoring and automated PDF report generation. The system is designed for roles such as Manual Testing, Automation Testing, AI Tech Support and Software Development. It is suitable for academic demonstration, internship review and first-level screening support.

The main contribution of this work is the integration of multiple simple but useful components into a single screening framework. Instead of treating interview evaluation as only an online test or only a video interview, the system combines identity verification, question assignment, answer scoring, camera status analysis, database persistence and report generation. This makes the

workflow easier to audit and easier to demonstrate in an academic environment.

Another contribution is the use of explainable scoring. Since first-level screening decisions should be understandable to evaluators, the system reports matched keywords, missing keywords, semantic similarity and communication score instead of producing only a hidden model output. This supports human review and helps the administrator understand why a candidate received a particular score.

II. RELATED WORK

Automated interview assessment has been studied using multiple data sources including verbal responses, facial cues, speech features and video behaviour. Naim et al. investigated automated prediction of interview performance using multimodal features. Nguyen and Gatica-Perez studied online video resumes and hirability impressions. Chen et al. explored automated judgement of video interviews using speech, facial and language characteristics. These works show that candidate assessment can benefit from computational analysis, but they often require large datasets and complex models.

NLP based answer assessment has also been widely used in educational and recruitment systems. Keyword matching and TF-IDF similarity provide explainable ways to compare candidate answers with expected responses. Scikit-learn provides practical tools for vectorization and cosine similarity. OpenCV supports real-time computer vision tasks such as face detection and frame analysis. AI recruitment studies also highlight the need for fairness, transparency and human oversight. The proposed system follows an explainable, low-cost approach by using classical NLP scoring and basic face monitoring rather than opaque decision-making models.

Existing recruitment tools usually fall into three categories: manual interview systems, online test portals and AI-enabled video interview platforms. Manual systems provide direct human judgement but require more time. Test portals are scalable but are usually limited to multiple-choice evaluation. Video interview platforms preserve richer candidate evidence but are often expensive and require manual review. The proposed system attempts to occupy a practical middle ground by providing a lightweight, locally deployable and explainable assessment process.

Approach	Strength	Limitation
Manual interview	Direct human judgement	High time and evaluator dependency
Online test portal	Fast objective scoring	Limited descriptive answer analysis
Video interview platform	Captures candidate behaviour	Often paid and review-heavy
Proposed system	Integrated NLP, CV, database and report flow	Basic AI methods; final HR judgement required

III. PROPOSED SYSTEM FRAMEWORK

The proposed system is organized as a layered web application. The presentation layer contains HTML, CSS and JavaScript pages

for registration, OTP verification, role selection, interview answering, camera capture and result display. The application layer is implemented using Flask routes and APIs. The AI evaluation layer performs answer scoring and camera-frame analysis. The data and report layer uses PostgreSQL for persistent records and ReportLab for PDF report generation.

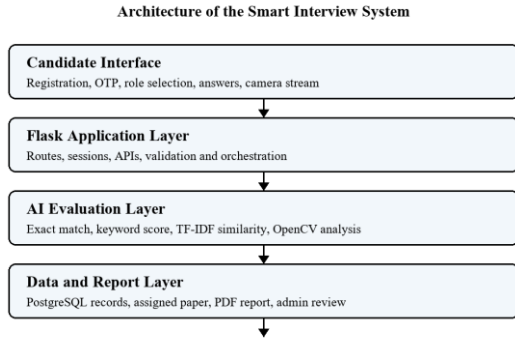


Fig. 1. Architecture of the proposed smart interview system.

A. Candidate Verification and Role Selection

The candidate first submits name, email, phone number and resume details. A one-time password is generated and verified before the candidate is allowed to continue. After successful verification, the candidate selects an interview role. The role determines the question pattern and the topics used for assessment.

B. Question Bank and Assignment

The question bank is stored in PostgreSQL. Each record contains role slug, section, topic, difficulty, question text, options, correct answer, keywords, marks and active status. When an interview starts, the system selects active questions for the selected role and records the exact assigned paper. This prevents later question-bank edits from changing completed interview records.

C. NLP Scoring Model

Objective answers are evaluated using exact answer matching. Descriptive answers are evaluated using keyword score, semantic similarity and communication score. The total descriptive score is calculated as a weighted combination of these three components:

$$\text{Total Score} = 0.35(\text{KS}) + 0.45(\text{TS}) + 0.20(\text{CS})$$

where KS denotes keyword score, TS denotes TF-IDF cosine similarity score and CS denotes communication score. This formula gives higher importance to semantic relevance while still rewarding important technical keywords and answer clarity.

D. Camera Monitoring

The browser captures webcam frames and sends encoded images to the Flask API. OpenCV converts the frame to grayscale and applies Haar cascade based face detection. The system records whether a face is detected, whether more than one face is present and whether the candidate remains centered. These signals are used as a basic confidence and monitoring indicator.

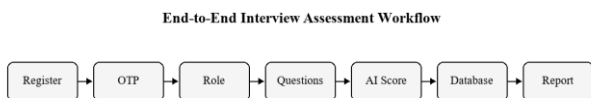


Fig. 2. End-to-end interview assessment workflow.

E. Report Generation and Administrator Review

After submission, the scoring results and camera summary are combined into a candidate-level output. The report contains candidate identity, selected role, total score, question-wise performance, feedback, confidence index and shortlist status. The report is stored in the project report directory and can be accessed

through the administrator report page. This reduces the time needed to manually prepare interview notes after every candidate attempt.

F. Algorithmic Flow

Step	Operation
1	Register candidate and verify OTP
2	Select role and fetch active questions from PostgreSQL
3	Store assigned paper for the interview session
4	Collect answers and camera frames during the test
5	Compute exact, keyword, TF-IDF and communication scores
6	Compute face presence and multiple-face summary
7	Generate result, shortlist decision and PDF report

IV. IMPLEMENTATION

The backend implementation is developed in Python using Flask. The main application file contains routes for home page, OTP verification, role selection, interview page, database view, admin reports, question API, frame analysis API, submit API, result page and report download. Utility modules are used for scoring, database operations, question-bank selection and PDF report generation.

The frontend uses HTML templates, CSS and JavaScript. The interview page loads questions, records answer selections, displays camera status, supports browser speech input and submits responses through an API call. The scoring module uses scikit-learn's TF-IDF vectorizer and cosine similarity. The database module uses PostgreSQL through pycogp. Report generation is handled using ReportLab.

The implementation follows a modular organization. The scoring functions are placed in a separate utility file so that the answer evaluation logic can be tested and improved independently. The database functions create and manage the tables required for roles, users, interviews, questions and candidate answers. The question-bank module is responsible for selecting the least-used active questions for a selected role. This separation improves maintainability and makes the application easier to extend.

Layer	Main Components	Responsibility
Frontend	HTML, CSS, JavaScript	Candidate forms, question display, camera capture and answer submission
Backend	Flask routes and APIs	Session handling, OTP workflow, validation and orchestration
AI Evaluation	Scikit-learn, OpenCV	NLP scoring, semantic similarity, face detection and alerts
Database	PostgreSQL	Question bank, users, interviews, assigned papers and answer records
Report	ReportLab	Score summary, feedback, confidence index and PDF report

The database design gives importance to auditability. The live question bank is editable, but the exact questions assigned to a candidate are copied into interview records. Therefore, if an administrator later edits or disables a question, the completed candidate record remains unchanged. This behaviour is important in interview systems because old results must remain traceable and consistent.

Data Entity	Purpose
users	Candidate identity, contact information, OTP status and resume path
roles	Supported interview roles and role names
question_bank	Editable active questions with topic, answer, keywords and marks
interviews	Interview id, candidate id, selected role, status and report path
candidate_answers	Submitted answers, AI score, feedback and keyword details

V. RESULTS AND DISCUSSION

The system was tested using sample candidate flows for role-based interviews. Candidate registration, OTP verification, role selection, question loading, answer submission, NLP scoring, face monitoring, database storage, PDF report generation and admin report viewing were verified. The intermediate implementation demonstrates approximately eighty percent completion of the overall project workflow.

For testing, candidate attempts were created for different roles. Each test run verified whether the expected questions were loaded, whether answers were accepted by the submit API, whether the score summary was generated and whether report paths were stored correctly. Camera analysis was tested by checking normal face presence, missing-face condition and multiple-face alert conditions. These tests confirmed that the major modules can interact as one end-to-end workflow.

Feature	Observed Result
OTP verification	Candidate can proceed only after verification
Role-wise questions	Questions are loaded from the PostgreSQL bank
NLP evaluation	Exact, keyword, TF-IDF and communication scores are generated
Camera monitoring	Face missing and multiple-face states are detected
Report generation	PDF report is created for administrator review

The key advantage of the system is that it provides a consistent screening process while keeping the scoring logic explainable. PostgreSQL based question management makes the question bank editable and reusable. The generated report reduces manual documentation effort. However, the system does not replace final human interview judgement. The current computer vision module provides only basic monitoring and the NLP model uses classical similarity techniques. Advanced transformer-based semantic scoring, dashboard analytics and secure cloud deployment can improve the system further.

A. Discussion

The observed behaviour shows that a classical NLP based approach can be useful when the goal is first-level screening rather than final selection. Exact answer matching works well for objective aptitude and programming questions. Keyword matching helps verify whether the candidate has mentioned important technical terms. TF-IDF cosine similarity gives an approximate semantic comparison between the submitted answer and the expected answer. Communication score rewards answers that are neither too short nor excessively unclear.

The OpenCV component is intentionally kept simple. It does not infer emotion or personality; it only provides a basic indication of whether a candidate is visible and whether more than one face appears in the frame. This design is safer for an academic project because it avoids overclaiming sensitive behavioural analysis while still adding a useful monitoring signal.

B. Practical Use

In a practical placement or internship screening scenario, the system can be used before a final technical interview. The administrator can prepare role-wise questions, share the test link, allow candidates to complete the assessment and then review generated reports. Candidates with stronger scores and acceptable monitoring status can be shortlisted for a human interview. This supports evaluators without removing human decision-making authority.

C. Limitations

The project has a few limitations. The descriptive scoring quality depends on the prepared ideal answer and keyword list. The system may not fully understand creative but correct answers if they are very different from the expected text. Camera monitoring depends on lighting, camera position and browser permission. Also, the current system does not include advanced authentication

such as government ID verification or plagiarism checking. These limitations can be addressed in future versions.

VI. CONCLUSION AND FUTURE WORK

This paper presented an AI-Based Smart Interview System for automated first-level candidate screening. The system integrates Flask, PostgreSQL, NLP scoring, OpenCV monitoring and PDF report generation into a single workflow. The proposed approach supports role-based assessment, transparent scoring and structured administrator review. It is suitable for academic and internship demonstration and can be extended for organizational screening support.

Future work includes adding HR login, improving semantic evaluation using BERT or sentence-transformer models, expanding the question bank, adding analytics dashboards, improving fairness checks and deploying the application on a secure server with stronger privacy controls.

The system can also be extended with candidate analytics such as role-wise average score, topic-wise weakness, repeated question usage and interview completion trends. A more mature version can include consent screens, privacy policy enforcement, access control, encrypted report storage and review dashboards. These enhancements would make the system more suitable for real-world pilot deployment.

REFERENCES

- [1] I. Naim et al., "Automated prediction and analysis of job interview performance," 2015.
- [2] L. Nguyen and D. Gatica-Perez, "Hirability in the wild: Analysis of online video resumes," 2016.
- [3] L. Chen et al., "Automated video interview judgment on large corpus," 2017.
- [4] S. Muralidhar et al., "Words worth: Verbal content and hirability impressions," 2018.
- [5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, 2011.
- [6] K. Sparck Jones, "A statistical interpretation of term specificity and its application in retrieval," 1972.
- [7] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.
- [8] J. Devlin et al., "BERT: Pre-training of deep bidirectional transformers for language understanding," 2019.
- [9] H. Suen et al., "Artificial intelligence in personnel selection," 2019.
- [10] N. Guenole et al., "Algorithmic recruitment and selection systems," 2022.